

The v0.7.2 decision

TOM v0.7.2 is a source-preserving extension of TOM v0.7. It takes the author's statement that every admitted definition is SDF-qualified as an operational closure rule: every named role and every compound expression must have an explicit signed-field operator. A role that has no numerical law does not disappear and does not receive a guessed floating formula. It evaluates to a typed symbolic field value or to an explicit admission status. The implementation now covers the complete native role set, including open Klein, Matryoshka, A, SELF and guarded Later_T terms, plus typed Observation and Actualization application terms.

This document therefore gives the undefined mechanics a place in the universal TOM: the operator registry, the typed field value, the temporal admission gate and the application-law boundary. It does not rewrite the original PDF, declare that the v0.7.1 recurrence was present in the source, or claim that a laboratory law has already been discovered.

InverseOccam[Arch[C[J[T]]; E[C[J[T]]; InvLog[phi]]]] qualified_by forward_T_precedence

The v0.7 parent remains the authority for the expression, lowercase phi, forward precedence, inverse-Occam non-erasure, TC1-TC5 and the rejection of a compulsory metric, coordinate grid, vector space or graph ontology. v0.7.2 adds a closure semantics and identifies which parts are intrinsic, which are an exact implementation profile and which must be supplied by a real-world application.

Fidelity to the v0.7 source

Source commitment	v0.7.2 treatment
Every admitted role is SDF-qualified	Made executable as a closure rule. Each operator returns a signed field term, a symbolic field term, or a declared rejection.
Lowercase phi and inverse-log role	Retained as typed roles. phi uses the exact Z[phi] algebra; InvLog[phi] is the symbolic constant kappa=-ln(phi).
Forward precedence TC1-TC5	Retained as the commit gate. Operators may derive a candidate, but only a valid later tick can publish it.
Inverse-Occam non-erasure	Retained as provenance and operand history. Equal readouts never replace the defining term.
No compulsory metric, grid, vector or graph	Retained. SDF means a definition-level signed field operator in this profile, not an imported Euclidean distance theorem.
Open jitter, pinion and echo mechanics	Given typed operators. Intrinsic closure leaves a missing law open; the default application profile selects v0.7.1's Thue-Morse and Z[phi] recurrence with its law named in the evidence.
Named open definitions and observation	Klein, Matryoshka, A, SELF and guarded Later_T remain literal SDF terms with obligations until explicitly bound. Observation and Actualization are typed application operators; no source law is invented.
Physical causality and entropy	Exposed as application-law slots. The kernel can preserve and test exact declared relations; it cannot turn a name into a measured physical constant.

The fidelity is high for binding, qualification, preservation and temporal order. It is an extension rather than a deduction for any concrete recurrence, measurement unit or physical prediction. The distinction is part of the formalization, not a disclaimer added after execution.

The universal signed-field term

A v0.7.2 field term is an exact, typed record. It is not required to be a point in a space. The sign describes the selected field relation; the magnitude or expression remains attached to its defining role.

FieldTerm = (value, sign, role, operands, unit, time, availability, provenance, history)

Component	Meaning
value	An exact integer, reduced rational, Z[phi] pair, or symbolic expression. Floats are not a value type in the deterministic profile.
sign	Negative, zero, positive or unknown. Unknown is preserved symbolically; it is never silently rounded or guessed.
role and operands	The named SDF operator and ordered source operands that produced the term.
unit	An application-declared exact unit tag. The tag is metadata and does not create a metric space.
time and availability	Integer experiment time and latest available evidence, checked by TC1 and TC5.
provenance and history	Source bytes, term identity, parent terms and committed prefix. This is the inverse-Occam requirement made inspectable.

The operator contract is total over the declared representation: an operator must return an admitted FieldTerm or an explicit status such as PROGRAM_ERROR, FUTURE_EVIDENCE or CAPACITY. An omitted branch is not a semantic wildcard.

The operator registry

Role	v0.7.2 operator contract
T	Read the current integer temporal state, delta, horizon and evidence bound. T never reads an unavailable future.
J[T]	Apply a declared exact jitter law to T. The intrinsic law is open; the current application profile selects $J(n)=(-1)^{\text{popcount}(n)}$, with the law retained in the term.
phi	Use the exact pinion algebra. For $C=a+b*\text{phi}$, $\text{phi}*C=(b)+(a+b)*\text{phi}$; no approximate square root is evaluated.
InvLog[phi]	Return the symbolic constant $\text{kappa}=-\ln(\text{phi})$, including its sign and source role. No decimal log is needed for exact execution.
C	Apply causal admissibility and the selected exact transition to an ordered jitter operand and the committed state.
E	Construct the ordered symbolic echo from the actual C result and InvLog[phi]. The echo is not silently renamed thermodynamic entropy.
Arch	Retain direct and echo terms, evaluate the declared Arch scalar only where its exact domain permits, and preserve operands when the readout is zero.
InverseOccam	Publish the field term with its source, operands and completed prefix intact. Applying the wrapper twice is the licensed idempotent qualification.
forward T	Commit the complete candidate only after all operators, evidence, time and capacity checks pass.
TOM, A, Klein, Matryoshka	Named declarations are literal SDF terms. TOM is a deferred source binding; A is open unless the explicit A=T alias is enabled; Klein and Matryoshka retain application-law obligations.
SELF and Later_T	SELF is a guarded deferred SDF term carrying its symbol id. Later_T carries its guard id and required later tick; it cannot synthesize an unguarded result.
Observation and Actualization	Application-bound SDF operators that retain sampled value, availability and commit/rejection status.

The registry is compositional but not a hidden graph ontology. Operand addresses are an external serialization of the ordered expression. The definition remains the expression and its licensed laws; the binary image is a reproducible representation.

Literal source SDFs and orientation

The raw source also names temporal, gradient, pinion, causal and echo SDFs. v0.7.2 carries each name as a literal operator or symbolic obligation, while keeping incompatible sign conventions separate instead of silently merging them.

Source SDF name	v0.7.2 treatment
SDF_t = t - t0	Exact integer temporal field when an application supplies t and t0; T enforces availability and forward publication.
J and grad J = 1	A named jitter field with a law slot. The Thue-Morse sign is the current discrete application profile; it is not presented as a proof of a continuous gradient theorem.
C orientation	A separate causal orientation law. The source's future-negative/past-positive convention is retained as metadata; an application must declare how its temporal sign maps to C.
phi scale and InvLog[phi]	Exact symbolic phi and kappa=-ln(phi); no decimal logarithm or approximate golden-ratio value is introduced.
E = kappa*C	Ordered symbolic echo. Its sign and compression interpretation remain tied to the selected C orientation and are not renamed thermodynamic entropy.
ArchU / Arch	Exact signed residual/readout where the selected operands and domain permit it. A zero residual preserves all operands and history.
Undefined divisions or zero-bound cases	Return an explicit open/rejected status with an obligation; never divide by zero or manufacture a continuation.

This is the literal implementation boundary: every source name has a field term, but a source name alone does not choose a metric, smoothness class, physical unit or empirical law.

One universal transition

candidate = forward_T(InverseOccam(Arch(C(J(T)); E(C(J(T)); InvLog(phi))))))

The transition first reads a committed snapshot. It derives a disposable candidate through the actual source-bound operators. If any operator cannot represent its exact result, the step stops without publishing a partial field. If all obligations pass, the complete value and its provenance become the next committed state.

Phase	Rule
Read	Use only the committed prefix and evidence available at the current integer time.
Derive	Evaluate every bound operator in source order, preserving ordered operands and symbolic constants.
Qualify	Require forward time, no future evidence, no erased prefix and declared capacity.
Commit	Append one full record and advance the state atomically. A failed later step leaves earlier committed steps intact.

This is how an undefined continuation becomes a mechanical choice without pretending that all real-world meanings are already known: the kernel has a universal typed operation boundary, while each application supplies an explicit law and an observable test.

Application laws and real-world physics

A physical application is a profile on top of the universal operator closure. It must declare what is measured, how measurements enter the signed field, and what observation could falsify the law. The kernel cannot infer these from the words causality or entropy.

Required application item	Exact declaration
Input	Sensor or experiment channel, provenance and availability time.
Representation	Unit tag and integer or reduced-rational encoding. Quantization is stated; no float is smuggled into the deterministic core.
Law	The ordered SDF operator expression and all parameters, including any replacement for a default v0.7.1 law.
Prediction	A future observable field term or bounded interval, with the relevant time horizon.
Falsifier	A result that would reject the application law while leaving the source definition intact.
Evidence	The raw input, transformation record and committed history needed to reproduce the decision.

This makes laws of physics possible as real applications without saying that the present GPU run already is physics. A successful application would require calibration, measurements, uncertainty bounds and repeatable disagreement tests outside the purely symbolic kernel.

Exact implementation profile

The v0.7.2 semantic reference is implemented in `python/tom/v072.py`. It evaluates every native source role as an `SDFValue`, keeps unresolved roles as first-class symbolic operators, and exposes exact Observation and Actualization records. `python/tom/v072_formal.py` composes the literal source into the native integer ABI and applies the complete transaction gate. The exact lattice application adapter is implemented in `python/tom/v072_lattice.py`. The existing v0.7.1 executable remains the first concrete GPU recurrence profile; v0.7.2 keeps its exact integer $Z[\phi]$ state and extends it with the registry and symbolic `FieldTerm` envelope. Existing v0.7.1 images remain valid and are interpreted as the default registry profile.

Property	Required v0.7.2 behavior
Determinism	Same source, registry, exact inputs and capacities produce the same output bytes and status.
No floats	No floating value type, float transition, approximate sqrt/log or float timing in authored compute paths.
Symbolic closure	A value that cannot be numerically reduced is retained as an expression with a known sign when available.
Capacity	Limb, expression, history and application limits are explicit. Exhaustion rejects the candidate rather than truncating it.
Compatibility	v0.7.1 default expression and vectors remain a required conformance profile.

The host reference, formal transaction layer and exact lattice application are implemented and independently tested. The native CUDA executor runs the finite formal ABI profile and matches the Python reference word for word. It is not claimed to accept arbitrary v0.7.2 lattice geometry or operator registries.

Validation and boundaries

The v0.7.2 validation in this release tests the source-bound expression, registry dispatch, symbolic preservation, exact signs, temporal contracts, non-erasure, guarded roles and an application manifest independently. The formal backend run covers 28 cases and compares 45,696 words on CPU and RTX 5070 Ti GPU with zero mismatches.

Gate	Evidence
Source fidelity	Original v0.7 PDF hash, expression and source bytes retained; changed bindings rejected.
Operator closure	Every registered role has a passing value or explicit status for normal, boundary and malformed inputs; open roles are tested as nonzero symbolic values.
Exactness	Authored values and PTX use no floats; symbolic kappa is retained without decimal substitution. Native SM120 SASS conversion helpers from integer division are reported separately, while the v0.7.1 recurrence has a strict no-FP audit.
Causality	Future evidence, retrograde time, erased prefix and failed atomic commits are rejected.
Compatibility	All v0.7.1 known vectors and resume archives reproduce exactly.
Application	The exact integer-line application manifest records input, law, predicted trace, falsifier and evidence digest; it is not confused with intrinsic TOM.

v0.7.2 does not prove an unbounded continuum, infinite memory, a specific topology for Klein or Matryoshka, thermodynamic entropy, or a physical law without the application evidence just described. It does provide a universal exact interface in which those laws can be stated without changing the source silently.

Consolidated statement

TOM v0.7.2 is the complete v0.7 expression under a universal SDF-operator closure. Every admitted role is represented as a signed, typed, provenance-preserving field term. Forward qualification governs publication; inverse-Occam prevents a readout from erasing its definition; exact symbolic values prevent floating approximation; and application manifests connect the operator algebra to real observations.

TOM_v0.7.2 := SDFOperatorClosure(TOM_v0.7) qualified_by forward_T_precedence

The closure is a formal extension, not a retroactive claim about the parent PDF. Its current executable profiles are the exact Python semantic layer, the native integer formal-transaction ABI, the integer-line application and the L-infinity lattice adapter; the v0.7.1 GPU recurrence remains the compatible accelerator profile. Future application laws may be richer, but each must name its added mechanics and provide a test that could fail.